

DApp Integration with Smart Contract

ALU Digital Asset Registry & Logo Token dApp — Project Report

Prepared by: Maxtoshie | African Leadership University (ALU) | Course: Programming / Blockchain Applications

Abstract

This project delivers a browser-based decentralized application (dApp) that sits in front of two Formative 1 smart contracts — **ALUAssetRegistry** (an ERC-721 registry for the official ALU logo) and **ALULogoToken** (an ERC-20 token representing 1,000,000 fractional ownership shares). The dApp lets any user connect a Web3 wallet, hash and register the logo directly in the browser, publicly verify whether a given file is the authentic logo with no wallet required, and lets the contract owner distribute ALUT ownership shares through a simple dashboard. It turns two otherwise-inaccessible smart contracts into a product any ALU student, faculty member, or administrator can actually use.

Introduction

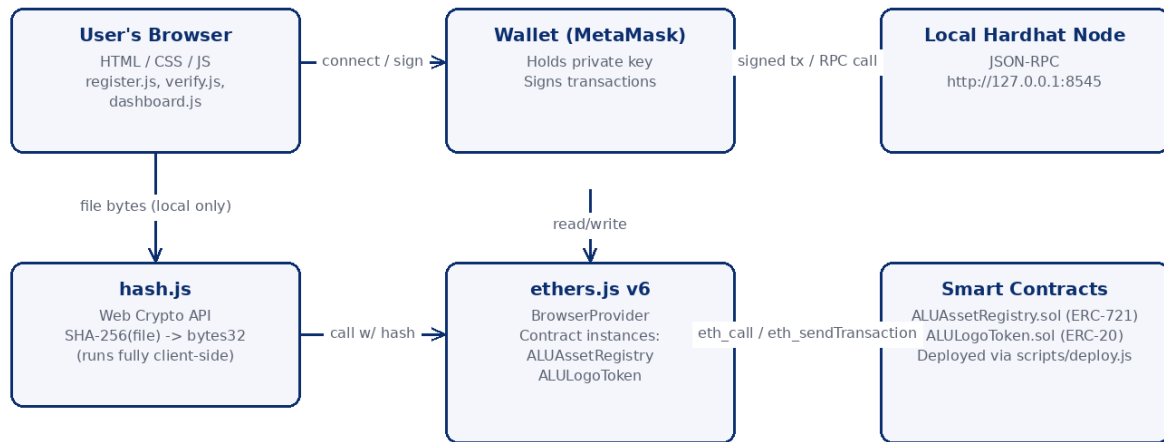
Formative 1 proved that the ALU logo could be registered and protected on-chain, and that ownership of it could be split into tradeable ERC-20 shares. But a smart contract by itself is not a usable product — nobody at ALU is going to open a terminal, install Hardhat, and run console commands just to check whether a logo file is authentic. The gap this dApp fills is exactly that last mile: a clean web interface that turns `registerAsset()`, `verifyLogoIntegrity()`, and `distributeShares()` into buttons, forms, and plain-language results that a non-technical person can use without ever knowing what a smart contract is.

Architecture Overview

The frontend is plain HTML/CSS/JavaScript (no build step) so it can be opened directly or served with a single static-file command. It uses **ethers.js v6** to talk to whichever wallet the browser has injected as `window.ethereum` (MetaMask in this project's testing). Two ethers.js Contract instances — one per smart

contract — are created once a wallet connects and are shared across the Register, Verify, and Dashboard tabs. The Verify page keeps its own read-only provider so it keeps working even with no wallet connected at all, since verification is a free, read-only blockchain call.

ALU dApp Architecture



Read calls (verifyLogoIntegrity, balanceOf, totalSupply) cost no gas and need no signature.
Write calls (registerAsset, distributeShares) require the wallet to sign and pay gas.

Figure 1 — Data flow between the browser, the wallet, ethers.js, and the two smart contracts.

Feature Walkthrough

A. Wallet Connection and Contract Integration

A prominent **Connect Wallet** button sits in the header on every tab. Once connected it is replaced by three pills showing the shortened wallet address (e.g. 0x1234...abcd), the connected wallet's current ALUT balance, and the active chain ID. If no wallet extension is detected, the app shows a clear inline message instead of failing silently. Account switching and network changes are handled through MetaMask's `accountsChanged` and `chainChanged` events.

[Insert screenshot here: the header before and after clicking "Connect Wallet", showing the address/balance/network pills.]

B. Logo Upload, In-Browser Hashing, and Registration

On the Register tab, selecting a file immediately shows an image preview and triggers `crypto.subtle.digest("SHA-256", ...)` in `hash.js`, which returns the hash formatted as a 0x-prefixed bytes32 string ready to pass straight into the contract. The user then fills in an asset name and file type and clicks **Register Asset**, which calls `registerAsset()`, prompts a MetaMask signature, and — once mined — reads the new token ID back out of the emitted `AssetRegistered` event to show a success message. Attempting to register a file whose hash is already on-chain surfaces the contract's revert reason as a friendly duplicate-hash message instead of a raw error.

[Insert screenshot here: file selected with preview + computed hash, then the success message with token ID.]

C. Public Logo Verification Page

The Verify tab needs no wallet at all. A user either uploads a file (hashed locally, then compared) or pastes a hash directly, together with the token ID to check against. The result is shown as a large, unmissable green "Logo Verified" panel or a red "Warning: This logo has been modified" panel — never a raw hex string — and, when the logo is authentic, the registered asset name, registration date, and registering wallet are shown underneath.

[Insert screenshot here: one successful ('Logo Verified') result and one failed ('Warning') result.]

D. Token Distribution Dashboard

The Dashboard tab shows the total ALUT supply, the connected wallet's balance and ownership percentage, and a table of three example holder addresses with their live balances and percentages. If — and only if — the connected wallet matches the contract's owner(), a Distribute Shares form appears, letting the owner send ALUT to any address (with client-side validation for a non-zero amount and sufficient balance before the transaction is even submitted). Any other wallet sees a note explaining that only the owner can distribute shares.

[Insert screenshot here: the ownership overview stats, the holders table, and the owner-only distribution form.]

Wallet and Web3 Integration

What is an ABI, and why does the frontend need it? The Application Binary Interface is the JSON description of a contract's functions, their parameter types, and its events. Solidity compiles down to raw EVM bytecode with no human-readable function names, so the ABI is what lets ethers.js encode a call like `registerAsset("ALU Logo", "png", 0x...)` into the correct byte-level calldata, and decode whatever comes back into JavaScript values the app can use.

Read-only call vs. transaction. A read-only call (like `verifyLogoIntegrity()` or `balanceOf()`) is answered directly by a node from its current state, costs no gas, and needs no signature — anyone can ask it. A transaction (like `registerAsset()` or `distributeShares()`) changes on-chain state, must be broadcast to the network, costs gas, and must be signed by the sender's private key before any node will accept it.

Why the wallet signs, not the app. The application never has access to the user's private key — it only ever has their public address. Signing happens inside the wallet extension precisely so that a malicious or buggy website can never move a user's funds or register assets on their behalf without their explicit, per-transaction approval.

Verification Page

Hashing happens with the Web Crypto API's `SubtleCrypto.digest()`, which runs entirely inside the browser sandbox; the file's bytes are read into memory with `File.arrayBuffer()` and never leave the device or touch a server. This matters for two reasons: it means the dApp needs no backend at all (and therefore nothing to hack, host, or trust), and it means a user never has to hand a potentially sensitive file to a third party just to check whether it is genuine — the comparison hash is computed locally and only that 32-byte hash, not the file, is ever sent anywhere.

Why no wallet is required. `verifyLogoIntegrity()` is declared as a Solidity view function — it only reads contract storage and never writes to it, so it costs zero gas and requires no signed transaction. Any Ethereum JSON-RPC node can answer it for free, which is why the Verify page works even for a visitor with no wallet installed at all.

Real-world scenario. Imagine a partner organisation receives an "ALU-branded" flyer from an unknown source and wants to confirm it's using the real logo before republishing it. They open the Verify page, upload the logo image extracted from the flyer, and get an immediate, plain-language answer — no blockchain knowledge, wallet, or ALU account required.

Why the result can be trusted. The comparison is not "ALU says so" — it's a deterministic hash comparison against data written immutably to a public blockchain at registration time. Anyone can independently re-run the same SHA-256 hash and the same contract call and get the identical answer; there is no central server whose response has to be taken on faith.

Token Dashboard

The dashboard reads `totalSupply()`, the connected wallet's `balanceOf()`, and `ownershipPercentage()` directly from `ALULogoToken`, so the numbers shown are always exactly what's on-chain — there is no cached or estimated data. Distribution is protected by Solidity's `onlyOwner` modifier: if any address other than the contract's stored owner calls `distributeShares()`, the transaction reverts before any state changes with an `OwnableUnauthorizedAccount` error, and no tokens move. The frontend mirrors that protection cosmetically by only rendering the distribution form for the owner's wallet, but the real security boundary is enforced on-chain regardless of what the frontend does or doesn't show.

Test Results

`test/ALUAssetRegistry.test.js` contains thirteen tests: the eight from Formative 1 covering the core contract behaviour, plus five new frontend-integration tests added for this assignment.

#	Test	Covers
1	Correct ERC-721 name/symbol	ALUAssetRegistry deployment
2	<code>registerAsset</code> mints token + stores metadata	Part B2
3	Duplicate hash is rejected	Part B2
4	<code>verifyLogoIntegrity</code> — matching hash	Part C

5	verifyLogoIntegrity — non-matching hash	Part C
6	Reverts on nonexistent token ID	Error handling
7	ALULogoToken mints 1,000,000 to owner	ALULogoToken deployment
8	distributeShares + onlyOwner + ownershipPercentage	Part D2
9	Frontend reads total ALUT supply as 1,000,000	New — Part D1
10	Hashing function returns correct bytes32 SHA-256	New — Part B1
11	Verification success path end-to-end	New — Part C1
12	Verification failure path end-to-end	New — Part C1
13	distributeShares updates recipient balance	New — Part D2

[Insert screenshot here: terminal output of `npm test` showing all 13 passing tests.]

Conclusion

Building this dApp made the gap between "a working smart contract" and "a usable product" very concrete: most of the effort here went into small trust and clarity details — clear error messages instead of raw revert strings, a visible preview before hashing a file, an unmissable verified/fake result, and hiding the owner-only form from everyone else — rather than into the contracts themselves, which barely changed from Formative 1. It was also a good exercise in the read-vs-write distinction in Web3: designing the Verify page to require no wallet at all only became obvious once the gas-cost implications of view functions were fully internalised.

With more time, the next additions would be: a proper WalletConnect/Coinbase Wallet adapter instead of only supporting browser-injected wallets, an indexed event feed (e.g. via a subgraph) to replace the hardcoded example-holders list with real live holder data, and a simple governance extension so ALUT holdings could back proportional voting rights, as discussed in the Part D report questions.

GitHub Repository

[Insert your public GitHub repository link here, e.g.
<https://github.com/<your-username>/alu-dapp-assignment2>]

Video Demonstration

[Insert a link to a short screen recording here showing: connecting a wallet, registering the logo, verifying it on the public page, and distributing ALUT shares from the owner wallet.]